

1. Introduction

1.1 Purpose

This document is the complete System Requirements Specification (SRS) and system design for **AgentHarness Enterprise** — a commercial, multi-tenant agentic AI orchestration platform evolved from AI_Workflow PAOS. It is designed to be sold as a GitHub repository (self-hosted) or a managed SaaS application that customers install and configure alongside their existing AI subscriptions (Claude, GPT-4, Gemini, local models). The platform provides a constitutional governance harness, cross-session continuity, pipeline orchestration, and a unified web dashboard — all working on top of whatever AI tools the customer already uses.

This SRS covers both the MVP (initial GitHub repo release) and the scalable enterprise evolution (multi-tenant SaaS, enterprise SSO, compliance exports). It is the primary design document for the productization of the PAOS concept.

1.2 Scope

AgentHarness Enterprise is a **Bring-Your-Own-AI** multi-agent orchestration harness that:

- Connects to any combination of AI providers (Anthropic Claude, OpenAI, Google Gemini, local Ollama models)
- Provides a constitutional governance layer (H-Factor protocol) that enforces separation of planning, review, and execution roles
- Maintains **cross-session continuity** — agents remember context across sessions, tools, and team members through a unified shared memory system
- Orchestrates multi-agent pipelines through a declarative `/h-pipeline` command and a visual pipeline builder
- Exposes all orchestration state through a web dashboard with team collaboration features
- Is installable from GitHub as a self-hosted Docker Compose application or available as a managed cloud service

In scope (MVP — GitHub self-hosted):

- Docker Compose deployment with web dashboard
- H-Factor constitutional governance engine
- Multi-agent roster with BYOA (Bring Your Own AI) connections
- Cross-session continuity via PostgreSQL-backed shared memory
- `/h-pipeline` cross-agent pipeline command
- Antigravity Review Loop (plan → review → execute → walkthrough)
- MCP server layer (shared-memory, scaffold)
- REST API for programmatic access
- Single-workspace (team) mode with basic user accounts
- Git-anchored audit trail
- Obsidian vault export compatibility

In scope (Enterprise / Scalable):

- Multi-tenant workspace isolation
- Enterprise SSO (SAML 2.0, OAuth 2.0, LDAP/AD)
- Role-based access control (RBAC) for team members

- Pipeline marketplace (installable skill/pipeline templates)
- Real-time agent activity feed via WebSockets
- Compliance exports (SOC 2, GDPR audit logs)
- Webhook integrations (GitHub, Jira, Slack, Linear)
- Self-hosted or cloud (AWS/GCP/Azure) managed deployment
- Custom agent plugin SDK
- Usage metering and billing (for SaaS tier)

Out of scope:

- Building or hosting the underlying AI models
- Replacing developer IDE tools (users keep their Claude Code, Cursor, etc.)
- Providing AI inference infrastructure (customers use their own API keys)

1.3 Definitions, Acronyms, Abbreviations

Term	Definition
AgentHarness	The product name for the enterprise evolution of PAOS
BYOA	Bring Your Own AI — customers connect their existing AI subscriptions
H-Factor	Hyper-Structured Factor — the constitutional governance protocol
MCP	Model Context Protocol — Anthropic's protocol for AI tool extensions
Cross-Continuity	The ability for agents to maintain and share context across sessions, tools, and team members
Pipeline	A structured plan-then-execute handoff from a planner agent to an executor agent
Harness	The governance + memory + orchestration layer that constrains and coordinates agents
Workspace	An isolated organizational unit (team, project, or company) in AgentHarness
Soul	An agent identity definition including role, permissions, and pipeline position
Antigravity Loop	The artifact-driven review cycle: TASKS.md → IMPLEMENTATION_PLAN.md → WALKTHROUGH.md
RBAC	Role-Based Access Control
SSO	Single Sign-On
SRS	System Requirements Specification
FR	Functional Requirement
NFR	Non-Functional Requirement
MVP	Minimum Viable Product

1.4 References

- IEEE Std 830-1998: Recommended Practice for Software Requirements Specifications

- AI_Workflow PAOS Constitution v2.1.0 (H-Factor Protocol) — design predecessor
- Anthropic Model Context Protocol Specification
- OpenAI API Documentation
- Google Gemini API Documentation
- NIST SP 800-53 Security Controls (for enterprise compliance)
- SOC 2 Type II Trust Services Criteria
- OAuth 2.0 RFC 6749
- SAML 2.0 Specification

1.5 Document Overview

Section	Content
1	Introduction
2	Literature Review
3	SWOT and PESTLE Analysis
4	Planning Phase
5	Methodology
6	Functional and Non-Functional Requirements
7	Process and Data Modelling
8	System Design
9	UML Diagrams
10	MVP Design
11	Testing
12	Deployment Plan
13	Maintenance Plan
14	References

\newpage

2. Literature Review

2.1 Existing Solutions

Tool	Strengths	Gap vs AgentHarness
LangChain / LangGraph	Mature Python ecosystem, graph-based agent flows	No cross-continuity, no governance constitution, SaaS-hostile (code-first)
AutoGen (Microsoft)	Multi-agent conversations, group chat patterns	No persistent memory, no pipeline marketplace, no BYOA model

CrewAI	Role-based crew definitions, task flows	Cloud-first, no file-system memory, no H-Factor governance
Agentflow / Flowise	Visual flow builders, no-code	No constitutional governance, no cross-tool agent identity, no compliance layer
Dust.tt	Team AI workflows, connector ecosystem	Closed platform, no self-host, no custom agent harness
Linear AI / GitHub Copilot Workspace	Issue-native AI agents	Single-tool scope, no cross-agent orchestration, no BYOA
Zapier AI Agents	Non-technical workflow automation	No coding capability, no audit trail, no governance
Dify	LLMOps platform, RAG + agent flows	No per-agent identity, no H-Factor governance, opinionated LLM routing
ControlFlow	Python-native structured agent tasks	Developer-only, no dashboard, no enterprise features

2.2 Gap Identified

No existing product combines: (1) a **constitutional governance engine** that enforces role separation and audit immutability across heterogeneous AI tools; (2) **BYOA connectivity** — the harness works with the AI subscriptions the customer already has (Claude, GPT-4, Gemini, local); (3) **cross-session continuity** — a shared memory system (PostgreSQL + optional Obsidian export) that persists agent context across sessions, users, and tools so work never starts from zero; (4) a **visual pipeline builder** that lets teams define plan-then-execute workflows without code; (5) **per-agent git identity** making every AI action attributable and auditable in the team's own repository; and (6) a **self-hosted GitHub repo** distribution model that gives enterprises full data sovereignty while optionally upgrading to a managed cloud tier.

2.3 Market Context

The enterprise AI tooling market is in the "picks and shovels" phase (2025–2027): companies have AI subscriptions but lack orchestration infrastructure. Research (a16z, Sequoia) consistently identifies multi-agent orchestration, audit trails, and AI governance as top enterprise pain points. AgentHarness addresses exactly these gaps with a developer-first, self-host-first product model that scales to SaaS for smaller teams.

2.4 Academic and Technical Foundations

Foundation	Application in AgentHarness
Blackboard Architecture (Engelmore, 1988)	PostgreSQL-backed shared memory hub
Constitutional AI (Anthropic, 2022)	H-Factor governance engine
CQRS + Event Sourcing	Append-only ledger + event stream
Role-Based Access Control (NIST, 2000)	Workspace member permissions

Choreography vs Orchestration (Peltz, 2003)	Hybrid: /h-pipeline = orchestration; inbox messaging = choreography
Saga Pattern (Garcia-Molina, 1987)	Pipeline execution with compensating transactions

\newpage

3. Analysis Frameworks

3.1 SWOT Analysis

	Positive	Negative
Internal	Strengths: Battle-tested PAOS architecture as foundation; BYOA avoids AI infrastructure costs; constitutional governance is a defensible moat; self-host model appeals to enterprise security requirements; Git-anchored audit trail is enterprise-familiar; MCP integration means compatibility with Claude ecosystem	Weaknesses: Setup complexity (requires Docker, API key configuration); MCP is Anthropic-specific (limits Google/OpenAI native support); file-system memory model must be migrated to DB for multi-user; no existing commercial customer base
External	Opportunities: Enterprise AI governance is a top-3 pain point (2025); Claude Max/Pro subscriptions growing rapidly; MCP ecosystem expanding; GitHub Marketplace distribution model validated; no direct constitutional governance competitor	Threats: Anthropic may ship native multi-agent orchestration; OpenAI Assistants API growing; large incumbents (Microsoft, Google) may ship enterprise agent platforms; customer resistance to another SaaS subscription

3.2 PESTLE Analysis

Factor	Analysis
Political	EU AI Act (2025) mandates audit trails for high-risk AI use; AgentHarness's append-only ledger is a compliance asset; US executive orders on AI safety favor auditable systems
Economic	Customers already pay for Claude/GPT subscriptions — BYOA model reduces barrier to adoption; SaaS pricing (per seat or per workspace) enables predictable revenue
Social	Engineering teams want AI that integrates with their existing workflow (Git, GitHub, Slack) rather than a new SaaS portal; developer-first positioning resonates
Technological	MCP adoption accelerating across AI tools; local model quality (Llama 3.3, Qwen 3) improving — makes BYOA local tier viable; WebSocket real-time UX now table-stakes
Legal	GDPR compliance required for EU customers; SOC 2 Type II certification needed for enterprise sales; self-host model gives customers data residency control
Environmental	Local model support reduces cloud API energy consumption; efficient pipeline routing reduces redundant AI calls

3.3 Business Model

Tier	Distribution	Price	Features
Open Core	GitHub (self-host, Docker)	Free	Full PAOS harness, 5 agents, unlimited pipelines, community support
Team	GitHub or cloud	\$49/workspace/month	25 agents, SSO, RBAC, webhook integrations, email support
Enterprise	Self-host or managed	Custom	Unlimited agents, SAML, LDAP, SOC 2, compliance exports, dedicated support
Cloud Managed	SaaS	\$29/seat/month	Fully managed, automatic updates, 99.9% SLA

\newpage

4. Planning Phase

4.1 Product Roadmap

Phase	Milestone	Timeline
Phase 0	Extract PAOS → AgentHarness Open Core repo	Q3 2026
Phase 1	Docker Compose MVP, PostgreSQL memory, REST API	Q4 2026
Phase 2	Web dashboard v2, pipeline builder, webhook integrations	Q1 2027
Phase 3	Team tier: SSO, RBAC, workspace isolation	Q2 2027
Phase 4	Enterprise tier: SAML, LDAP, SOC 2, compliance exports	Q3 2027
Phase 5	Cloud managed SaaS, billing, usage metering	Q4 2027
Phase 6	Pipeline Marketplace, agent plugin SDK	Q1 2028

4.2 Target Customers

Segment	Profile	Pain Point
Developer Teams (5–50)	Eng teams with Claude/GPT subscriptions, building with AI	No coordination layer between AI tools; no audit trail; context lost between sessions
AI-Forward Enterprises	Large orgs deploying AI coding assistants at scale	Governance, compliance, access control, auditability
AI Consultancies	Agencies building AI workflows for clients	Need repeatable harness they can deploy per-client

Solo AI Power Users	Individual developers/researchers	Open Core tier; same PAOS experience they know
---------------------	-----------------------------------	--

4.3 Stakeholders

Stakeholder	Role	Interest
Abdullah Abdul Hakim	Founder / Product Owner	Product direction, IP, revenue
NodeAlgo	Company	Business development, licensing
Enterprise Customers	Primary buyers	Governance, compliance, productivity
Developer Teams	End users	UX, integration quality, reliability
AI Providers (Anthropic, OpenAI, Google)	Ecosystem partners	BYOA compatibility, MCP compliance
GitHub	Distribution channel	Marketplace listing, repo stars

\newpage

5. Methodology

5.1 Development Approach

AgentHarness is developed using **PAOS itself** as the development harness — a recursive bootstrap where the tool is built by the tool. All feature development goes through H-Factor 3-phase pipelines, ensuring governance is continuously tested.

5.2 Architecture Philosophy

Principle	Application
Self-hosted first	Docker Compose is the primary deployment artifact; cloud is an optional wrapper
BYOA	No AI inference — we route to customer’s own API keys
File-system compatible	PostgreSQL stores normalized data; all artifacts exportable as markdown for Obsidian
API-first	REST + WebSocket API first; dashboard is a first-party client of the API
Open Core	Constitutional governance, MCP server, pipeline engine are open source; enterprise features are licensed
Git-anchored	All agent actions result in attributable git commits

5.3 Technology Stack Decision

Layer	Technology	Rationale
Backend API	Node.js (TypeScript) + Fastify	Consistent with MCP servers; fast I/O; strong ecosystem
Database	PostgreSQL	ACID, JSON columns for flexible metadata, proven at scale
Memory Cache	Redis	Session state, real-time event queuing
Frontend	Next.js (TypeScript) + Tailwind	Proven in PAOS dashboard; SSR for SEO
MCP Layer	Node.js (existing)	Reuse shared-memory and scaffold servers
Agent Communication	MCP + REST webhooks	MCP for tool calls; webhooks for external integrations
Auth	NextAuth.js + Passport	OAuth 2.0, SAML 2.0, LDAP
Git Integration	isomorphic-git / nodegit	Per-agent commits from server
Deployment	Docker Compose (self-host), ECS/Cloud Run (managed)	Single-command setup for self-host
CI/CD	GitHub Actions	Automated testing, release builds
Observability	OpenTelemetry + Grafana	Metrics, traces, log aggregation

\newpage

6. Functional and Non-Functional Requirements

6.1 Functional Requirements

6.1.1 Agent Harness Engine

ID	Requirement	Priority	Tag
FR-AH-01	The system shall maintain a registry of all agents in a workspace with: id, label, role, AI provider binding, permissions, inbox, log path, git identity	Must	MVP
FR-AH-02	Each agent shall have a unique identity (name + email) used for git commits	Must	MVP
FR-AH-03	The system shall support connecting agents to external AI providers: Anthropic Claude, OpenAI GPT, Google Gemini, and local Ollama models via API key configuration	Must	MVP

FR-AH-04	The system shall enforce H-Factor role boundaries: Planner ≠Reviewer ≠Executor ≠Orchestrator (I1)	Must	MVP
FR-AH-05	The system shall support at minimum 8 built-in agent role templates: coordinator, planner, architect, developer, reviewer, notifier, researcher, summarizer	Must	MVP
FR-AH-06	Operators shall be able to create custom agents by defining a soul (role, permissions, AI provider, pipeline position) via the web dashboard	Should	MVP
FR-AH-07	The system shall provide an Agent Plugin SDK allowing third-party agents to be packaged and installed from the Pipeline Marketplace	Could	Scalable
FR-AH-08	Agent health status (online/offline/error) shall be tracked and surfaced in the dashboard in real time	Should	Scalable

6.1.2 Cross-Continuity Engine

ID	Requirement	Priority	Tag
FR-CC-01	The system shall persist all agent shared memory in PostgreSQL so context survives server restarts, session changes, and tool switches	Must	MVP
FR-CC-02	The system shall maintain a per-workspace global ledger (append-only event log) with all agent actions across all sessions	Must	MVP
FR-CC-03	At session start, any agent shall be able to retrieve: last N ledger entries, current shared context, its inbox messages, and the most recent HANDOFF state — in one API call	Must	MVP
FR-CC-04	The system shall maintain a HANDOFF document per workspace, rewritten at each session end, that contains: current in-progress tasks, blocked items, key decisions, and next recommended actions	Must	MVP
FR-CC-05	The system shall support cross-tool continuity: when a user switches from Claude Code to Codex to Gemini, each tool reads the same shared memory and picks up where the previous left off	Must	MVP
FR-CC-06	The system shall maintain a per-agent event stream so any agent's full action history is queryable	Must	MVP
FR-CC-07	The system shall support Obsidian vault export: all shared memory artifacts exportable as markdown files compatible with the Obsidian desktop application	Should	MVP
FR-CC-	The system shall support automatic session summarization: when a session ends, the agent's actions are summarized and appended to the	Should	MVP

08	shared context		
FR-CC-09	Cross-continuity shall work across team members: if user A hands off to user B, user B's agent starts with full context from user A's session	Must	Scalable
FR-CC-10	The system shall detect context drift (when an agent's working assumption conflicts with newer ledger entries) and alert the active agent	Could	Scalable

6.1.3 Pipeline Orchestration

ID	Requirement	Priority	Tag
FR-PO-01	The system shall support the /h-pipeline command from any connected agent: submit a plan+tasks package to an executor agent's inbox	Must	MVP
FR-PO-02	Pipeline submission shall atomically create: pipeline package (PLAN + TASKS + META), task card, inbox message, ledger entry	Must	MVP
FR-PO-03	Pipelines shall have a configurable executor agent; default shall be the workspace's designated developer agent	Must	MVP
FR-PO-04	The system shall support the Antigravity Review Loop: Phase 1 (plan artifacts) → Phase 2 (user/agent review) → Phase 3 (execute) → Phase 4 (walkthrough)	Must	MVP
FR-PO-05	Execution shall be gated on an Architect PASS or CONDITIONAL review token	Must	MVP
FR-PO-06	The system shall provide a visual pipeline builder in the dashboard: drag-and-drop agent roles, define handoff points, set executor and reviewer	Should	Scalable
FR-PO-07	The system shall support parallel pipeline execution: multiple PIPE-xxx packages running concurrently without state collision	Should	Scalable
FR-PO-08	The system shall support pipeline templates installable from the Pipeline Marketplace	Could	Scalable
FR-PO-09	Failed pipelines shall support retry with exponential backoff; blocked pipelines shall notify the operator via webhook/email	Should	Scalable
FR-PO-10	The system shall track pipeline SLA: time from submission to completion, per agent role time, and flag overruns	Could	Scalable

6.1.4 Shared Memory and Knowledge

ID	Requirement	Priority	Tag
FR-SM-01	The system shall expose a shared memory MCP server providing all 12 tool operations to any connected agent	Must	MVP
FR-SM-02	The global ledger shall be backed by PostgreSQL (append-only table) with full-text search	Must	MVP
FR-SM-03	The task board shall support YAML-defined task cards with the full PAOS status flow	Must	MVP
FR-SM-04	Agent inboxes shall be backed by PostgreSQL with delivery guarantees (messages persist until explicitly acknowledged)	Must	MVP
FR-SM-05	The knowledge base shall support full-text search across all .md documents	Should	MVP
FR-SM-06	The scaffold MCP server shall provide template listing and project generation from the knowledge/templates directory	Must	MVP
FR-SM-07	The system shall support knowledge base imports: users can upload .md, .pdf, or .txt files that are indexed and searchable by agents	Should	Scalable
FR-SM-08	The system shall support RAG (Retrieval-Augmented Generation) over the knowledge base: agents can query the knowledge base via vector similarity	Could	Scalable

6.1.5 Web Dashboard

ID	Requirement	Priority	Tag
FR-DB-01	The system shall provide a web dashboard accessible at the configured host:port	Must	MVP
FR-DB-02	The dashboard shall display: live ledger feed, task board, agent roster, inbox, pipeline status, and handoff state	Must	MVP
FR-DB-03	The dashboard shall provide real-time updates via WebSocket — ledger entries and task status changes appear without page reload	Must	MVP
FR-DB-04	The dashboard shall allow operators to create tasks, send inbox messages, trigger pipelines, and review artifacts	Must	MVP
FR-DB-05	The dashboard shall include a visual pipeline builder for defining agent workflows	Should	Scalable

FR-DB-06	The dashboard shall include a knowledge base browser with full-text search	Should	Scalable
FR-DB-07	The dashboard shall show per-agent activity timelines and health status	Should	Scalable
FR-DB-08	The dashboard shall support dark/light themes and mobile-responsive layout	Should	MVP

6.1.6 Authentication and Authorization

ID	Requirement	Priority	Tag
FR-AA-01	The system shall support email + password authentication for self-hosted deployments	Must	MVP
FR-AA-02	The system shall support OAuth 2.0 login (GitHub, Google)	Must	MVP
FR-AA-03	The system shall support SAML 2.0 for enterprise SSO	Must	Scalable
FR-AA-04	The system shall support LDAP/Active Directory integration	Should	Scalable
FR-AA-05	The system shall enforce workspace-level RBAC with roles: Owner, Admin, Member, Viewer	Must	Scalable
FR-AA-06	API keys for AI providers shall be stored encrypted at rest (AES-256)	Must	MVP
FR-AA-07	Agent actions shall be attributable to the authenticated user who initiated the session	Must	MVP

6.1.7 API and Integrations

ID	Requirement	Priority	Tag
FR-API-01	The system shall expose a REST API for all core operations: agents, tasks, ledger, pipelines, inbox, memory	Must	MVP
FR-API-02	The system shall expose WebSocket endpoints for real-time event streaming	Must	MVP
FR-API-03	The system shall support outbound webhooks: pipeline completion, task status change, agent error	Should	MVP
FR-API-04	The system shall integrate with GitHub: create issues, PRs, and comments from pipeline artifacts	Should	Scalable
FR-API-05	The system shall integrate with Slack: send pipeline notifications and receive task commands	Should	Scalable

FR-API-06	The system shall integrate with Linear: create issues and track pipeline-to-issue mapping	Could	Scalable
FR-API-07	The system shall integrate with Jira: bidirectional task sync	Could	Scalable
FR-API-08	The REST API shall use JWT authentication with configurable token expiry	Must	MVP

6.1.8 Compliance and Audit

ID	Requirement	Priority	Tag
FR-CA-01	All agent actions shall be recorded in an immutable audit log with: timestamp, agent identity, action type, affected files/resources, status	Must	MVP
FR-CA-02	The system shall support audit log export in CSV, JSON, and NDJSON formats	Should	Scalable
FR-CA-03	The system shall support configurable data retention policies (e.g., keep audit logs for 365 days)	Should	Scalable
FR-CA-04	The system shall generate SOC 2 compliance reports on demand	Could	Scalable
FR-CA-05	The system shall support GDPR data export (all data for a user) and GDPR deletion requests	Must	Scalable
FR-CA-06	Sensitive data (API keys, secrets) shall never appear in audit logs or ledger entries	Must	MVP

6.2 Non-Functional Requirements

6.2.1 Performance

ID	Requirement	Tag
NFR-P-01	MCP tool calls shall complete within 200ms at p95 under 100 concurrent agents	MVP
NFR-P-02	Pipeline submission shall complete within 1 second under normal load	MVP
NFR-P-03	Dashboard initial load shall complete within 2 seconds	MVP
NFR-P-04	WebSocket event delivery shall occur within 500ms of event creation	MVP
NFR-P-05	Full-text search over 100,000 ledger entries shall complete within 1 second	Scalable
NFR-P-06	The system shall support 1,000 concurrent WebSocket connections per server instance	Scalable

6.2.2 Scalability

ID	Requirement	Tag
NFR-SC-01	The system shall support horizontal scaling of API servers behind a load balancer	Scalable
NFR-SC-02	PostgreSQL shall support read replicas for query scaling	Scalable
NFR-SC-03	The system shall support 10,000 agents across all workspaces in a multi-tenant deployment	Scalable
NFR-SC-04	The ledger shall support 1,000,000 entries without performance degradation	Scalable

6.2.3 Reliability

ID	Requirement	Tag
NFR-R-01	The system shall achieve 99.9% uptime SLA for the managed cloud tier	Scalable
NFR-R-02	The global ledger shall be append-only — no entry shall ever be modified or deleted	MVP
NFR-R-03	Failed MCP tool calls shall return structured errors within 5 seconds	MVP
NFR-R-04	The system shall perform automated PostgreSQL backups every 6 hours	Scalable
NFR-R-05	Pipeline execution shall be idempotent — retrying a failed pipeline shall not create duplicate artifacts	Scalable

6.2.4 Security

ID	Requirement	Tag
NFR-S-01	All API keys shall be stored encrypted (AES-256-GCM) in the database	MVP
NFR-S-02	All API communication shall use TLS 1.3	MVP
NFR-S-03	JWT tokens shall expire within 24 hours; refresh tokens within 30 days	MVP
NFR-S-04	Workspace data shall be isolated at the database row level (tenant_id on all tables)	Scalable
NFR-S-05	The system shall pass OWASP Top 10 security assessment	Scalable

NFR-S-06	Admin actions shall require MFA	Scalable
----------	---------------------------------	----------

6.2.5 Interoperability

ID	Requirement	Tag
NFR-I-01	The MCP server shall comply with Anthropic MCP specification v1.0+	MVP
NFR-I-02	Obsidian vault exports shall be compatible with Obsidian Desktop v1.5+	MVP
NFR-I-03	Git commits shall be compatible with GitHub, GitLab, and Gitea	MVP
NFR-I-04	REST API shall follow OpenAPI 3.1 specification	MVP

\newpage

7. Process and Data Modelling

7.1 Context Diagram (Level 0 DFD)

```
flowchart LR
    OP([Team Members\n/ Operators])
    AH[AgentHarness\nEnterprise]
    CLAUDE([Anthropic\nClaude API])
    OPENAI([OpenAI\nGPT API])
    GOOGLE([Google\nGemini API])
    OLLAMA([Ollama\nLocal Models])
    GITHUB([GitHub\nRepo])
    SLACK([Slack\nWorkspace])
    OBS([Obsidian\nVault Export])

    OP <-->|Web Dashboard\n/ MCP / API| AH
    AH <-->|BYOA API Calls| CLAUDE
    AH <-->|BYOA API Calls| OPENAI
    AH <-->|BYOA API Calls| GOOGLE
    AH <-->|Local Inference| OLLAMA
    AH -->|Agent Commits| GITHUB
    AH -->|Pipeline Notifications| SLACK
    AH -->|Vault Export| OBS
```

7.2 Level 1 DFD — Core Processes

```
flowchart LR
    subgraph AH["AgentHarness Enterprise"]
        P1[1. Harness\nEngine]
        P2[2. Cross-Continuity\nEngine]
        P3[3. Pipeline\nOrchestrator]
    end
```

```

P4[4. MCP\nGateway]
P5[5. Auth &\nWorkspace]
P6[6. Dashboard\n& API]
DS1[(PostgreSQL\nShared Memory)]
DS2[(Redis\nEvent Cache)]
DS3[(File Storage\nArtifacts)]
end

OP([Operator]) --> P6
P6 --> P1
P6 --> P3
P1 <--> P2
P3 <--> P2
P4 <--> P2
P2 <--> DS1
P2 <--> DS2
P3 <--> DS3
P5 --> P1
P5 --> P6

```

7.3 Entity Relationship Diagram

```

erDiagram
    WORKSPACE {
        uuid id PK
        string name
        string slug
        string plan
        datetime created_at
    }
    USER {
        uuid id PK
        uuid workspace_id FK
        string email
        string role
        datetime created_at
    }
    AGENT {
        uuid id PK
        uuid workspace_id FK
        string name
        string role
        string provider
        string model_alias
        string git_email
        string inbox_channel
        string risk_level
    }
    AI_PROVIDER {
        uuid id PK
    }

```



```

        uuid workspace_id FK
        string provider_name
        string api_key_encrypted
        string[] model_ids
    }
    LEDGER_ENTRY {
        uuid id PK
        uuid workspace_id FK
        uuid agent_id FK
        uuid user_id FK
        datetime timestamp
        string action
        string resource
        string status
        jsonb metadata
    }
    TASK_CARD {
        uuid id PK
        uuid workspace_id FK
        uuid agent_id FK
        string title
        string status
        jsonb frontmatter
        datetime created_at
        datetime updated_at
    }
    PIPELINE {
        uuid id PK
        uuid workspace_id FK
        uuid planner_agent FK
        uuid executor_agent FK
        string status
        text plan_content
        text tasks_content
        datetime submitted_at
        datetime completed_at
    }
    INBOX_MESSAGE {
        uuid id PK
        uuid workspace_id FK
        uuid from_agent FK
        uuid to_agent FK
        string subject
        text body
        boolean acknowledged
        datetime sent_at
    }
    HANDOFF {
        uuid id PK
        uuid workspace_id FK
        text content
        datetime updated_at
    }

```

```

        uuid updated_by FK
    }
    SHARED_CONTEXT {
        uuid id PK
        uuid workspace_id FK
        uuid agent_id FK
        text entry
        datetime appended_at
    }

    WORKSPACE ||--o{ USER : "has"
    WORKSPACE ||--o{ AGENT : "has"
    WORKSPACE ||--o{ AI_PROVIDER : "configures"
    WORKSPACE ||--o{ LEDGER_ENTRY : "has"
    WORKSPACE ||--o{ TASK_CARD : "has"
    WORKSPACE ||--o{ PIPELINE : "has"
    WORKSPACE ||--o{ INBOX_MESSAGE : "has"
    WORKSPACE ||--|| HANDOFF : "has"
    WORKSPACE ||--o{ SHARED_CONTEXT : "has"
    AGENT ||--o{ LEDGER_ENTRY : "produces"
    AGENT ||--o{ TASK_CARD : "owns"
    AGENT ||--o{ PIPELINE : "plans or executes"
    AGENT ||--o{ INBOX_MESSAGE : "sends or receives"
    AGENT --> AI_PROVIDER : "binds to"

```

7.4 Pipeline State Machine

```

stateDiagram-v2
    [*] --> submitted : /h-pipeline called
    submitted --> plan_produced : Planner writes PLAN + TASKS
    plan_produced --> awaiting_review : Sent to Architect inbox
    awaiting_review --> review_pass : PASS token
    awaiting_review --> review_fail : FAIL token
    review_fail --> plan_produced : Plan revised
    review_pass --> awaiting_execution : Handoff to executor inbox
    awaiting_execution --> executing : Executor picks up
    executing --> completed : All tasks done
    executing --> blocked : Error / dependency
    blocked --> awaiting_execution : Retry
    completed --> committed : agent-commit.sh
    committed --> [*]

```

7.5 Cross-Continuity Data Flow

```

flowchart TD
    S1[Session Starts\nAny Agent / Any Tool]
    S1 --> R1[Read HANDOFF\nfrom PostgreSQL]
    S1 --> R2[Read last 20 ledger rows\nMCP read_ledger]

```

```

S1 --> R3[Read inbox messages\nMCP read_inbox]
R1 & R2 & R3 --> SYNTH[Agent synthesizes\nfull context]
SYNTH --> WORK[Agent works]
WORK --> L1[After each action:\nMCP append_ledger]
WORK --> L2[After key decision:\nMCP write_context]
WORK --> L3[Write agent events\nto event stream]
L1 & L2 & L3 --> END[Session Ends]
END --> HW[Rewrite HANDOFF\nwith current state]
HW --> SC[Append session summary\nto shared context]
SC --> COM[agent-commit.sh]
COM --> NEXT[Next agent / session\nstarts from full context]

```

\newpage

8. System Design

8.1 High-Level Architecture

flowchart TD

```

    subgraph ClientLayer["Client Layer"]
        BROWSER[Web Browser\nDashboard]
        CLAUDE_EXT[Claude Code\nIDE Extension]
        CODEX[Codex CLI]
        GEMINI[Gemini CLI]
        OTHER[Other MCP\nClients]
    end

    subgraph APILayer["API Layer (Node.js / Fastify)"]
        REST[REST API\n/api/v1/*]
        WS[WebSocket\n/ws/events]
        MCP_GW[MCP Gateway\n:n:stdio or :http]
    end

    subgraph ServiceLayer["Service Layer"]
        HARNESS[Harness Engine\nAgent lifecycle]
        CONTINUITY[Cross-Continuity\nEngine]
        PIPELINE[Pipeline\nOrchestrator]
        AUTH[Auth Service\nJWT / SSO]
        WEBHOOK[Webhook\nDispatcher]
    end

    subgraph StorageLayer["Storage Layer"]
        PG[(PostgreSQL\nPrimary DB)]
        REDIS[(Redis\nEvent Cache)]
        S3[(S3/MinIO\nArtifact Storage)]
        GIT[Git\nRepo]
    end

    subgraph ExternalLayer["External Layer"]

```

```

    CLAUDE_API[Anthropic API]
    OPENAI_API[OpenAI API]
    GOOGLE_API[Google API]
    OLLAMA_LOCAL[Ollama Local]
    GITHUB_API[GitHub API]
    SLACK_API[Slack API]
end

ClientLayer <--> APILayer
APILayer <--> ServiceLayer
ServiceLayer <--> StorageLayer
ServiceLayer <--> ExternalLayer

```

8.2 Cross-Continuity Engine Design

The Cross-Continuity Engine is the core differentiator of AgentHarness. It solves the "blank slate" problem — where every AI session starts with no memory of prior work.

8.2.1 Continuity Layers

Layer	Storage	Scope	Lifetime
Session State	Redis	Single session	Session duration
HANDOFF Document	PostgreSQL	Workspace	Until next session end
Shared Context	PostgreSQL (append-only)	Workspace	Permanent
Global Ledger	PostgreSQL (append-only)	Workspace	Permanent
Agent Event Stream	PostgreSQL	Per-agent	Permanent
Task Board	PostgreSQL	Workspace	Until done/archived
Pipeline Packages	S3/MinIO	Per-pipeline	Configurable retention
Knowledge Base	PostgreSQL + Vector DB	Workspace	Permanent

8.2.2 Session Bootstrap API

A single API call at session start loads full continuity context:

```
GET /api/v1/workspaces/{id}/continuity?agent={agent_id}&limit=20
```

Response:

```

{
  "handoff": { "content": "...", "updated_at": "..." },
  "ledger": [ { "timestamp", "agent", "action", "resource", "status" }, ... ],
  "inbox": [ { "from", "subject", "body", "sent_at" }, ... ],
  "in_progress_tasks": [ { "task_id", "title", "status" }, ... ],
  "blocked_tasks": [ ... ]
}

```

8.2.3 Cross-Tool Continuity Flow

```
sequenceDiagram
    participant T1 as Claude Code (User A)
    participant CC as Cross-Continuity Engine
    participant PG as PostgreSQL
    participant T2 as Codex (User B)

    T1->>CC: Session Start (workspace_id, agent=claude)
    CC->>PG: SELECT handoff + ledger + inbox
    PG-->>CC: Full context
    CC-->>T1: Bootstrap context
    T1->>T1: Works on task
    T1->>CC: append_ledger(entry)
    CC->>PG: INSERT ledger_entry
    T1->>CC: Session End – rewrite HANDOFF
    CC->>PG: UPDATE handoff SET content=...
    Note over T2: Later – different user, different tool
    T2->>CC: Session Start (workspace_id, agent=codex)
    CC->>PG: SELECT handoff + ledger + inbox
    PG-->>CC: Full context INCLUDING User A's work
    CC-->>T2: Bootstrap context
    Note over T2: Codex picks up exactly where Claude left off
```

8.3 H-Factor Governance Engine

The governance engine enforces constitutional invariants as middleware in the API and MCP layers.

8.3.1 Invariant Enforcement

Invariant	Enforcement Point	Mechanism
I1 Separation of Powers	Pipeline submit	Reject pipelines where planner = reviewer or reviewer = executor
I2 Audit Immutability	Ledger API	Database trigger prevents UPDATE/DELETE on ledger_entries table
I3 Identity First	All API calls	JWT claim + agent registry lookup required; anonymous actions rejected
I4 Skill Boundary	MCP tool calls	Tool call authorization matrix checked against agent's declared permissions

8.3.2 Governance Middleware Stack

Request → Auth Middleware → Identity Resolution → Permission Check → Action → Ledger Append → Response

8.4 MCP Gateway Design

AgentHarness exposes the MCP server as both a stdio process (for IDE integrations like Claude Code) and an HTTP endpoint (for remote/cloud deployments).

```
flowchart LR
    CL_EXT[Claude Code\nExtension]
    CD_CLI[Codex CLI]
    GM_CLI[Gemini CLI]

    subgraph MCPGateway["MCP Gateway"]
        STDIO[stdio transport\nlocal process]
        HTTP[HTTP/SSE transport\ncloud deployment]
        ROUTER[Tool Router]
        SM_TOOLS[shared-memory\ntools x12]
        SC_TOOLS[scaffold\ntools x2]
        CUSTOM[Custom Agent\nPlugin Tools]
    end

    CL_EXT --> STDIO
    CD_CLI --> STDIO
    GM_CLI --> HTTP
    STDIO --> ROUTER
    HTTP --> ROUTER
    ROUTER --> SM_TOOLS
    ROUTER --> SC_TOOLS
    ROUTER --> CUSTOM
    SM_TOOLS --> PG[(PostgreSQL)]
    SC_TOOLS --> FS[(File Storage)]
```

8.5 Pipeline Marketplace Design

The Pipeline Marketplace is an installable registry of pipeline templates, agent soul definitions, and skill packages.

```
flowchart LR
    MPK[Pipeline\nMarketplace\nGitHub Registry]
    AH[AgentHarness\nEnterprise]
    INSTALL[Install\nCommand]
    TMPL[Pipeline\nTemplate]
    SOUL[Agent\nSoul]
    SKILL[Skill\nPackage]

    MPK --> INSTALL
    INSTALL --> AH
    AH --> TMPL
    AH --> SOUL
    AH --> SKILL
```

Marketplace entry format (marketplace.yaml):

```

name: github-issue-pipeline
version: 1.2.0
description: Routes GitHub issues through PAOS planning and execution pipeline
author: nodealgo
components:
  - type: pipeline-template
    file: pipeline.yaml
  - type: agent-soul
    file: souls/github-agent.md
  - type: skill
    file: skills/github-integration/SKILL.md

```

8.6 Database Schema

Core Tables

```

-- Workspaces (multi-tenant root)
CREATE TABLE workspaces (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name TEXT NOT NULL,
  slug TEXT UNIQUE NOT NULL,
  plan TEXT NOT NULL DEFAULT 'open-core',
  created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Agents
CREATE TABLE agents (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  workspace_id UUID REFERENCES workspaces(id) ON DELETE CASCADE,
  name TEXT NOT NULL,
  role TEXT NOT NULL,
  provider TEXT NOT NULL,
  model_alias TEXT,
  git_name TEXT NOT NULL,
  git_email TEXT NOT NULL UNIQUE,
  permissions JSONB NOT NULL DEFAULT '{}',
  created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Global Ledger (append-only – UPDATE/DELETE denied by trigger)
CREATE TABLE ledger_entries (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  workspace_id UUID REFERENCES workspaces(id),
  agent_id UUID REFERENCES agents(id),
  user_id UUID REFERENCES users(id),
  timestamp TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  action TEXT NOT NULL,
  resource TEXT,
  status TEXT NOT NULL CHECK (status IN ('OK', 'FAIL', 'CONDITIONAL')),
  metadata JSONB DEFAULT '{}'
);

```

```

);
CREATE INDEX idx_ledger_workspace_time ON ledger_entries(workspace_id, timestamp
DESC);

-- Immutability trigger
CREATE RULE no_update_ledger AS ON UPDATE TO ledger_entries DO INSTEAD NOTHING;
CREATE RULE no_delete_ledger AS ON DELETE TO ledger_entries DO INSTEAD NOTHING;

-- Pipelines
CREATE TABLE pipelines (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  workspace_id UUID REFERENCES workspaces(id),
  pipe_code TEXT UNIQUE NOT NULL, -- PIPE-001, PIPE-002, etc.
  planner_agent_id UUID REFERENCES agents(id),
  executor_agent_id UUID REFERENCES agents(id),
  reviewer_agent_id UUID REFERENCES agents(id),
  status TEXT NOT NULL DEFAULT 'submitted',
  plan_content TEXT,
  tasks_content TEXT,
  review_token TEXT,
  submitted_at TIMESTAMPTZ DEFAULT NOW(),
  completed_at TIMESTAMPTZ
);

-- Task Cards
CREATE TABLE task_cards (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  workspace_id UUID REFERENCES workspaces(id),
  pipeline_id UUID REFERENCES pipelines(id),
  title TEXT NOT NULL,
  status TEXT NOT NULL DEFAULT 'proposed',
  frontmatter JSONB DEFAULT '{}',
  created_at TIMESTAMPTZ DEFAULT NOW(),
  updated_at TIMESTAMPTZ DEFAULT NOW()
);

-- Handoff (one per workspace)
CREATE TABLE handoffs (
  workspace_id UUID PRIMARY KEY REFERENCES workspaces(id),
  content TEXT NOT NULL DEFAULT '',
  updated_at TIMESTAMPTZ DEFAULT NOW(),
  updated_by UUID REFERENCES agents(id)
);

```

8.7 REST API Design

Method	Path	Description
GET	/api/v1/workspaces/{id}/continuity	Session bootstrap — full context
GET	/api/v1/workspaces/{id}/ledger	Paginated ledger entries

POST	/api/v1/workspaces/{id}/ledger	Append ledger entry
GET	/api/v1/workspaces/{id}/tasks	List task cards
POST	/api/v1/workspaces/{id}/tasks	Create task card
PATCH	/api/v1/workspaces/{id}/tasks/{id}	Update task status
GET	/api/v1/workspaces/{id}/pipelines	List pipelines
POST	/api/v1/workspaces/{id}/pipelines	Submit pipeline
GET	/api/v1/workspaces/{id}/pipelines/{id}	Get pipeline detail
GET	/api/v1/workspaces/{id}/inbox/{agent_id}	Read agent inbox
POST	/api/v1/workspaces/{id}/inbox	Send inbox message
GET	/api/v1/workspaces/{id}/handoff	Get current handoff
PUT	/api/v1/workspaces/{id}/handoff	Rewrite handoff
GET	/api/v1/workspaces/{id}/agents	List agents
POST	/api/v1/workspaces/{id}/agents	Register agent
GET	/api/v1/workspaces/{id}/context	Get shared context
POST	/api/v1/workspaces/{id}/context	Append context entry
POST	/api/v1/workspaces/{id}/commits	Execute agent commit

\newpage

9. UML Diagrams

9.1 System Use Case Diagram

```
flowchart LR
    OP([Operator / Team\nMember])
    CAI([Claude Code\nAgent])
    DEV([Developer\nAgent])
    AR([Architect\nAgent])
    CO([Coordinator\nAgent])
    ADM([Workspace\nAdmin])

    OP --> UC1[Submit Task]
    OP --> UC2[Invoke /h-pipeline]
    OP --> UC3[View Dashboard]
    OP --> UC4[Review Pipeline Artifacts]
    OP --> UC5[Search Knowledge Base]
    CAI --> UC6[Load Session Context\nCross-Continuity]
    CAI --> UC7[Plan with Antigravity Loop]
```

```

CAI --> UC2
CAI --> UC8[Append Ledger]
CAI --> UC9[Send Inbox Message]
DEV --> UC10[Execute Approved Plan]
DEV --> UC6
DEV --> UC8
AR --> UC11[Review Plan PASS/FAIL]
AR --> UC6
CO --> UC12[Route Pipeline]
CO --> UC13[Verify Completion]
ADM --> UC14[Manage Agents]
ADM --> UC15[Configure AI Providers]
ADM --> UC16[Set RBAC Roles]
ADM --> UC17[Export Audit Logs]
UC1 --> UC7
UC7 --> UC11
UC11 --> UC10
UC10 --> UC13

```

9.2 Component Diagram

```

flowchart LR
    subgraph Clients["Client Layer"]
        WEB[Web Dashboard]
        IDE[IDE Extensions\nClaude / Codex / Gemini]
    end
    subgraph API["API Layer"]
        REST[REST API]
        WSS[WebSocket Server]
        MCPGW[MCP Gateway]
    end
    subgraph Services["Service Layer"]
        HARN[Harness Engine]
        CONT[Cross-Continuity Engine]
        PIPE[Pipeline Orchestrator]
        AUTH[Auth Service]
        HOOK[Webhook Dispatcher]
    end
    subgraph Storage["Storage"]
        PG[(PostgreSQL)]
        REDIS[(Redis)]
        MINIO[(MinIO / S3)]
        GITREPO[Git Repository]
    end
    subgraph External["External AI"]
        ANTHROPIC[Anthropic\nClaude]
        OPENAI[OpenAI\nGPT]
        GOOGLE[Google\nGemini]
        OLLAMA[Ollama\nLocal]
    end

```

```
Clients <--> API
API <--> Services
Services <--> Storage
HARN <--> External
HOOK --> External
```

9.3 Sequence Diagram — Cross-Continuity Handoff

```
sequenceDiagram
    participant UA as User A (Claude Code)
    participant UB as User B (Codex)
    participant CCE as Cross-Continuity Engine
    participant PG as PostgreSQL
    participant GIT as Git Repo

    UA->>CCE: Session Start (workspace, agent=claude)
    CCE->>PG: Bootstrap query (handoff + ledger + inbox)
    PG-->>CCE: Context package
    CCE-->>UA: Full session context
    UA->>UA: Executes tasks
    UA->>CCE: append_ledger x N
    CCE->>PG: INSERT ledger_entries
    UA->>CCE: agent_commit("Agent[claude]: ...")
    CCE->>GIT: git commit --author="Claude Code <claude@paos>"
    UA->>CCE: Session End – rewrite HANDOFF
    CCE->>PG: UPDATE handoffs SET content=...
    CCE->>PG: INSERT shared_context entry
    Note over UB: Next session – different user, different tool
    UB->>CCE: Session Start (workspace, agent=codex)
    CCE->>PG: Bootstrap query
    PG-->>CCE: Context INCLUDING User A's HANDOFF + ledger
    CCE-->>UB: User B has full context of User A's work
    UB->>UB: Continues exactly where User A left off
```

9.4 Deployment Diagram — Self-Hosted

```
flowchart TD
    subgraph Docker["Docker Compose (Customer Server)"]
        subgraph App["Application Containers"]
            API["API Server\nFastify :3001"]
            DASH["Next.js Dashboard\n:3333"]
            MCP["MCP Gateway\n:3002"]
        end
        subgraph Data["Data Containers"]
            PG["PostgreSQL\n:5432"]
            REDIS["Redis\n:6379"]
            MINIO["MinIO\n:9000"]
        end
    end
```

```

    end
    subgraph Agent["Agent Processes"]
        OLLAMA[Ollama\nLocal LLM]
    end
end
subgraph External["External Services"]
    CLAUDE_API[Anthropic Claude API]
    OPENAI_API[OpenAI API]
    GITHUB[GitHub]
    SLACK[Slack]
end
subgraph Client["Client Machines"]
    BROWSER[Browser\nDashboard]
    IDE[VS Code +\nClaude Code ext]
end

BROWSER --> DASH
IDE --> MCP
DASH --> API
MCP --> API
API --> PG
API --> REDIS
API --> MINIO
API --> CLAUDE_API
API --> OPENAI_API
API --> GITHUB
API --> SLACK
OLLAMA --> API

```

9.5 Deployment Diagram — Managed Cloud (Enterprise SaaS)

```

flowchart TD
    subgraph AWS["AWS / GCP / Azure"]
        subgraph LB["Load Balancer"]
            ALB[Application\nLoad Balancer]
        end
        subgraph AppTier["Application Tier (Auto-Scaling)"]
            API1[API Server 1]
            API2[API Server 2]
            DASH1[Dashboard 1]
        end
        subgraph DataTier["Data Tier"]
            PGPRIMARY[(PostgreSQL\nPrimary)]
            PGREPLICA[(PostgreSQL\nRead Replica)]
            REDIS_CLUSTER[(Redis\nCluster)]
            S3[(S3\nArtifact Storage)]
        end
        subgraph Infra["Infrastructure"]
            CDN[CloudFront / CDN]
            WAF[WAF / DDoS Protection]
        end
    end

```

```

        end
    end
    subgraph Monitoring["Observability"]
        GRAFANA[Grafana]
        OTEL[OpenTelemetry\nCollector]
    end
    end

    CUSTOMER([Enterprise\nCustomers]) --> CDN
    CDN --> WAF
    WAF --> ALB
    ALB --> AppTier
    AppTier --> DataTier
    AppTier --> OTEL
    OTEL --> GRAFANA

```

9.6 Class Diagram — Core Domain

```

classDiagram
    class Workspace {
        +UUID id
        +String name
        +String slug
        +String plan
        +Agent[] agents
        +addAgent(soul) Agent
        +getBootstrapContext(agentId) ContinuityContext
    }
    class Agent {
        +UUID id
        +String name
        +String role
        +String provider
        +String gitEmail
        +Permissions permissions
        +loadContext() ContinuityContext
        +appendLedger(entry) void
        +sendMessage(to, body) void
        +agentCommit(message) void
    }
    class ContinuityContext {
        +Handoff handoff
        +LedgerEntry[] recentLedger
        +InboxMessage[] inbox
        +TaskCard[] inProgressTasks
        +summarize() String
    }
    class Pipeline {
        +UUID id
        +String pipeCode
        +Agent planner
    }

```

```

    +Agent executor
    +Agent reviewer
    +String status
    +PlanDocument plan
    +TasksDocument tasks
    +submit() void
    +applyReviewToken(token) void
    +execute() void
    +complete() void
}
class HarnessEngine {
    +registerAgent(soul) Agent
    +validatePipeline(pipeline) bool
    +enforceI1(planner, reviewer, executor) bool
    +enforceI4(agent, toolCall) bool
}
class CrossContinuityEngine {
    +bootstrapSession(workspaceId, agentId) ContinuityContext
    +appendLedger(entry) void
    +writeContext(entry) void
    +rewriteHandoff(content) void
    +detectContextDrift(agent) Alert[]
}
class GovernanceMiddleware {
    +checkIdentity(request) AgentIdentity
    +checkPermission(agent, action) bool
    +enforceImmutability(resource) void
    +logViolation(invariant, agent) void
}

Workspace "1" --> "many" Agent
Workspace --> CrossContinuityEngine
Workspace --> HarnessEngine
Agent --> ContinuityContext
Pipeline --> Agent
HarnessEngine --> GovernanceMiddleware
CrossContinuityEngine --> ContinuityContext

```

9.7 Activity Diagram — Antigravity Review Loop (Enterprise)

flowchart TD

```

    START([Team Member submits task]) --> ASSIGN[System assigns Planner Agent\nbased on workspace defaults]
    ASSIGN --> P1[Phase 1: Planner produces\nTASKS.md + IMPLEMENTATION_PLAN.md]
    P1 --> NOTIFY1[Notify team via WebSocket\nDashboard shows new artifacts]
    NOTIFY1 --> REVIEW{User reviews\nartifacts}
    REVIEW -->|Feedback| REVISE[Planner revises artifacts]
    REVISE --> NOTIFY1
    REVIEW -->|APPROVED| ARCH[Route to Architect Agent inbox]
    ARCH --> ARCHREVIEW{Architect reviews}

```

```
ARCHREVIEW -->|FAIL| PM_REWISE[PM revises plan]
PM_REWISE --> ARCH
ARCHREVIEW -->|PASS| SUBMIT[submit_pipeline to executor]
SUBMIT --> EXEC[Developer Agent executes\nPhase 3: runs each task]
EXEC --> UPDATE[Update task card status\nAppend ledger after each task]
UPDATE --> DONE{All tasks done?}
DONE -->|No| EXEC
DONE -->|Yes| WALKTHROUGH[Phase 4: Produce WALKTHROUGH.md]
WALKTHROUGH --> COMMIT[agent-commit.sh]
COMMIT --> NOTIFY2[Notify team: pipeline complete]
NOTIFY2 --> END([Done])
```

\newpage

10. MVP Design

10.1 MVP Feature Set

The AgentHarness Enterprise MVP is a single-workspace deployment installable from GitHub via Docker Compose.

Feature	MVP	Scalable Enterprise
H-Factor governance engine	✓	✓
BYOA: Anthropic Claude	✓	✓
BYOA: OpenAI GPT	✓	✓
BYOA: Google Gemini	✓	✓
BYOA: Ollama local	✓	✓
PostgreSQL shared memory	✓	✓
Cross-session continuity	✓	✓
Cross-tool continuity	✓	✓
MCP server (stdio + HTTP)	✓	✓
/h-pipeline command	✓	✓
Antigravity Review Loop	✓	✓
Web dashboard	✓	✓
REST API (OpenAPI 3.1)	✓	✓
WebSocket real-time feed	✓	✓
Outbound webhooks	✓	✓
Email + password auth	✓	✓

OAuth 2.0 (GitHub, Google)	✓	✓
Obsidian vault export	✓	✓
Git per-agent commit	✓	✓
Single workspace	✓	—
Multi-tenant workspaces	—	✓
SAML 2.0 / LDAP SSO	—	✓
RBAC (Owner/Admin/Member/Viewer)	—	✓
SOC 2 compliance exports	—	✓
Visual pipeline builder	—	✓
Pipeline Marketplace	—	✓
Agent Plugin SDK	—	✓
Horizontal API scaling	—	✓
Managed cloud SaaS	—	✓
GitHub / Slack / Linear integrations	—	✓
RAG over knowledge base	—	✓
Context drift detection	—	✓
Usage metering / billing	—	✓

10.2 MVP Architecture

```

flowchart LR
    subgraph Docker["Docker Compose"]
        API[Fastify API\n:3001]
        DASH[Next.js\n:3333]
        MCP[MCP Gateway\n:3002]
        PG[(PostgreSQL\n:5432)]
        REDIS[(Redis\n:6379)]
        MINIO[(MinIO\n:9000)]
    end

    CL_EXT[Claude Code\nIDE] --> MCP
    BROWSER[Browser] --> DASH
    DASH --> API
    MCP --> API
    API --> PG
    API --> REDIS
    API --> MINIO

```



```
API --> CLAUDE[Anthropic\nAPI]
API --> OPENAI[OpenAI\nAPI]
```

10.3 GitHub Repository Structure

```
agentharness/
├─ README.md
├─ docker-compose.yaml
├─ docker-compose.enterprise.yaml
├─ .env.template
├─ api/                                ← Fastify REST + WebSocket API (TypeScript)
│   ├── src/
│   │   ├── routes/
│   │   ├── services/
│   │   │   ├── harness-engine.ts
│   │   │   ├── cross-continuity.ts
│   │   │   ├── pipeline-orchestrator.ts
│   │   │   └── governance-middleware.ts
│   │   └── db/
│   │       └── migrations/
│   └── package.json
├─ dashboard/                          ← Next.js web dashboard (TypeScript)
│   ├── app/
│   └── package.json
├─ mcp/                                ← MCP Gateway (Node.js)
│   ├── shared-memory-server/
│   └── scaffold-server/
├─ harness/                            ← Core H-Factor governance engine
│   ├── constitution/
│   │   └── workflow.md
│   ├── agents/
│   │   └── registry.json
│   └── skills/
├─ plugins/                            ← Agent Plugin SDK
│   └── sdk/
├─ docs/                              ← Documentation
│   ├── quick-start.md
│   ├── configuration.md
│   ├── byoa-setup.md
│   └── api-reference.md
├─ scripts/                           ← Operational scripts
│   ├── agent-commit.sh
│   └── health-check.sh
└─ examples/                          ← Example agent souls + pipeline templates
```

10.4 Quick Start (Self-Hosted)

```
# 1. Clone
git clone https://github.com/nodealgo/agentharness
```

```
cd agentharness

# 2. Configure
cp .env.template .env
# Edit .env: set ANTHROPIC_API_KEY, OPENAI_API_KEY, etc.

# 3. Launch
docker compose up -d

# 4. Access dashboard
open http://localhost:3333

# 5. Connect Claude Code
# Add to ~/.claude/mcp.json:
# { "servers": { "agentharness": { "url": "http://localhost:3002/mcp" } } }
```

\newpage

11. Testing

11.1 Testing Strategy

Level	Method	Tooling
Unit	Service function tests, governance middleware	Vitest (TypeScript)
Integration	API routes + PostgreSQL	Supertest + Testcontainers
MCP	Tool call contract tests	MCP test client
End-to-end	Full /h-pipeline flow	Playwright
Cross-continuity	Multi-session context handoff	Scripted scenarios
Cross-tool	Claude → Codex continuity	Integration test suite
Performance	100 concurrent agents	k6 load testing
Security	OWASP Top 10	OWASP ZAP + manual
Governance	H-Factor invariant enforcement	Automated invariant tests

11.2 Requirements Traceability Matrix

FR ID	Test Case	Method	Pass Criteria
FR-CC-03	Session bootstrap returns handoff + ledger + inbox in one call	Integration	Response contains all three sections; < 200ms
FR-CC-05	Cross-tool continuity: Claude → Codex	E2E scenario	Codex session bootstrap includes Claude's HANDOFF

FR-AH-04	H-Factor I1 enforcement: planner ≠ executor	Governance test	Pipeline with same agent as planner + executor rejected
FR-PO-05	Execution blocked without PASS token	Integration	POST /pipelines/{id}/execute returns 403 without review token
FR-AA-02	OAuth 2.0 GitHub login	E2E	User authenticates, JWT issued, session established
FR-CA-01	Ledger entries immutable	DB test	UPDATE/DELETE on ledger_entries fails at DB level
FR-PO-02	Pipeline submission atomic	Integration	All 5 artifacts created or none (transaction rollback on failure)
FR-DB-03	WebSocket delivers ledger event within 500ms	Performance	Event measured from DB insert to WebSocket receive
NFR-P-01	MCP tool calls < 200ms p95	Load test	k6 p95 < 200ms at 100 concurrent
NFR-S-01	API keys encrypted at rest	Security	DB column shows AES-256-GCM ciphertext, not plaintext

11.3 Governance Invariant Tests

```
describe('H-Factor Governance', () => {
  it('I1: rejects pipeline where planner = executor', async () => {
    const result = await api.post('/pipelines', {
      planner_agent_id: 'agent-A',
      executor_agent_id: 'agent-A', // same as planner – violates I1
      reviewer_agent_id: 'agent-B'
    });
    expect(result.status).toBe(400);
    expect(result.body.error).toContain('I1: Separation of Powers');
  });

  it('I2: ledger entries cannot be deleted', async () => {
    const entry = await db.query("DELETE FROM ledger_entries WHERE id = $1",
[testId]);
    expect(entry.rowCount).toBe(0); // rule prevents deletion
  });

  it('I3: anonymous actions rejected', async () => {
    const result = await api.post('/ledger').withoutAuth();
    expect(result.status).toBe(401);
  });

  it('I4: agent cannot call tools outside declared permissions', async () => {
    const result = await mcpClient.callTool('scaffold_project', {}, { agent:
'viewer-agent' });
    expect(result.error).toContain('I4: Skill Boundary');
  });
});
```

```
});  
});
```

\newpage

12. Deployment Plan

12.1 Self-Hosted (Docker Compose)

Target: Developer teams, enterprises with data sovereignty requirements

```
# Production deployment  
docker compose -f docker-compose.yaml \  
               -f docker-compose.prod.yaml \  
               up -d  
  
# Services started:  
# - api:3001 (Fastify REST + WebSocket)  
# - dashboard:3333 (Next.js)  
# - mcp:3002 (MCP Gateway)  
# - postgres:5432  
# - redis:6379  
# - minio:9000
```

Environment configuration (`.env`):

```
# AI Providers (BYOA)  
ANTHROPIC_API_KEY=sk-ant-...  
OPENAI_API_KEY=sk-...  
GOOGLE_API_KEY=...  
  
# Database  
POSTGRES_URL=postgresql://user:pass@postgres:5432/agentharness  
REDIS_URL=redis://redis:6379  
  
# Auth  
JWT_SECRET=...  
GITHUB_CLIENT_ID=...  
GITHUB_CLIENT_SECRET=...  
  
# Storage  
MINIO_ENDPOINT=minio:9000  
MINIO_ACCESS_KEY=...  
MINIO_SECRET_KEY=...  
  
# Git  
GIT_REPO_PATH=/workspace/repo
```

12.2 Managed Cloud (Enterprise SaaS)

Target: Teams without infrastructure expertise, sub-50 seat deployments

- Single-click deployment via AWS Marketplace / GCP Marketplace
- Automated SSL certificate provisioning
- Managed PostgreSQL (RDS / Cloud SQL)
- Managed Redis (ElastiCache / Memorystore)
- S3 for artifact storage
- Daily automated backups with point-in-time recovery
- 99.9% SLA with auto-scaling

12.3 GitHub Marketplace Listing

- **Repository:** `github.com/nodealgo/agentharness`
- **Topics:** `ai-agents`, `multi-agent`, `mcp`, `claude`, `llm-orchestration`, `developer-tools`
- **License:** Business Source License (BSL 1.1) — open for <100 agents; enterprise license above
- **Marketplace:** GitHub App listing for OAuth integration
- **Docs site:** `docs.agentharness.dev`

\newpage

13. Maintenance Plan

13.1 Release Cadence

Type	Frequency	Scope
Patch	Weekly	Bug fixes, security patches
Minor	Monthly	New features, integrations
Major	Quarterly	Breaking changes, new tiers

13.2 Upgrade Path for PAOS Users

Existing PAOS operators can migrate to AgentHarness Enterprise in three steps:

1. **Import:** Run `agentharness import --from-paos ~/AI_Workflow/` — imports ledger, agents, tasks, and vault content to PostgreSQL
2. **Reconfigure:** Update AI tool MCP configs to point to AgentHarness MCP endpoint (`:3002`) instead of local stdio processes
3. **Validate:** Run `agentharness health-check` to verify agent connectivity, ledger integrity, and continuity

All existing HANDOFF.md, global_ledger.md, task cards, and pipeline artifacts are preserved.

13.3 Open Core Contribution Model

Component	License	Contribution
-----------	---------	--------------

H-Factor governance engine	Apache 2.0	Open contributions
MCP server (shared-memory, scaffold)	Apache 2.0	Open contributions
Antigravity Review Loop skill	Apache 2.0	Open contributions
Pipeline Orchestrator	BSL 1.1	Contributor License Agreement
Enterprise auth (SAML, LDAP)	Commercial	NodeAlgo only
Multi-tenant isolation	Commercial	NodeAlgo only
Managed cloud tier	Commercial	NodeAlgo only

13.4 SLA and Support

Tier	SLA	Support
Open Core	None	GitHub Issues
Team	99.5% / 48h response	Email
Enterprise	99.9% / 4h response	Dedicated Slack + email
Managed Cloud	99.9% / 1h response	24/7 on-call

\newpage

14. References

1. H-Factor Protocol v2.1.0 — ~/AI_Workflow/workflow.md (design predecessor)
2. SRS-1-PAOS-Current-State — companion SRS document (this folder)
3. IEEE Std 830-1998: IEEE Recommended Practice for Software Requirements Specifications. IEEE, 1998.
4. Anthropic. *Model Context Protocol Specification*. 2024.
5. Anthropic. *Constitutional AI: Harmlessness from AI Feedback*. 2022.
6. OpenAI. *Assistants API Documentation*. 2025.
7. Google DeepMind. *Gemini API Documentation*. 2025.
8. NIST SP 800-53 Rev 5: Security and Privacy Controls for Information Systems. NIST, 2020.
9. SOC 2 Trust Services Criteria. AICPA, 2022.
10. RFC 6749: The OAuth 2.0 Authorization Framework. IETF, 2012.
11. SAML 2.0 Technical Overview. OASIS, 2008.
12. Garcia-Molina, H., & Salem, K. (1987). Sagas. *ACM SIGMOD Record*, 16(3), 249–259.
13. Peltz, C. (2003). Web services orchestration and choreography. *IEEE Computer*, 36(10), 46–52.
14. Englemore, R., & Morgan, T. (Eds.). *Blackboard Systems*. Addison-Wesley, 1988.
15. Business Source License 1.1. MariaDB Corporation, 2023.
16. OWASP Top 10. Open Web Application Security Project, 2021.
17. OpenTelemetry Specification. CNCF, 2024.
18. OpenAPI 3.1 Specification. OpenAPI Initiative, 2021.